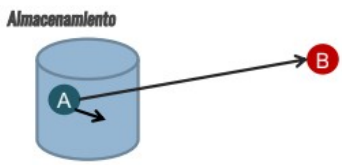
Clase 1

Persistencia por alcance

Establece que todo objeto volátil “B” al que se puede llegar a partir de un objeto persistente “A”, debe ser necesariamente persistente.

Se trata de abstraer las operaciones directas con la BD de la app, es decir, desligar la BD y tratar los objetos como elementos de la app en la RAM.




```
public class A{
  protected B b;

  public void setB(B aB){
    this.b=aB;
  }
}
```

```
A a=//obtengo “a” desde la BD

a.setB(new B());
```

- El obj. A debe estar persistido
- el obj. B debe estar relacionado con A (ej: por referencia o colección)

Las operaciones en cascadas están relacionadas a cómo quiero manejar la PxA.

- Cuando se realiza una operación CRUD sobre un objeto, esa misma operación se propaga automáticamente a los objetos relacionados.

Número mínimo de save()

Solo se necesita un save() en el objeto raíz (RootObject). La PxA asegura que todos los objetos relacionados se persistan automáticamente cuando se guarda el RootObject.

Ej: en un Banco, al hacer un save() del objeto Banco, solo debo vincular todas mis clases a ese objeto, y no se necesitaría de otro save().

Despersistencia por alcance

Es la posibilidad de que un objeto deje de ser persistente y se elimine automáticamente de la base de datos cuando todos los objetos que lo referencian rompen sus vínculos con él. Esto se logra mediante la configuración de cascadas, en particular, aquellas que permiten la eliminación de huérfanos.

El atributo que **fuerza la desvinculación** de un objeto persistente es el que habilita la eliminación de huérfanos, y en la mayoría de los ORMs o sistemas de persistencia se llama **orphanRemoval**=true.

Este atributo le dice al sistema: Si este objeto deja de estar vinculado desde su padre, entonces lo elimina de la base.

Persistencia Ortogonal

Principio de ortogonalidad de tipos

Establece que cada objeto, independiente de su tipo, tiene derecho a ser persistido. Este principio tiene la relevancia de asegurar que no haya discriminación en la persistencia de objetos basada en su tipo, lo que facilita la extensibilidad y mantenimiento del código.

Principio de independencia de la persistencia

La introducción de persistencia (por ej: guardar datos en una base) no debería cambiar la forma en que se ve el código. El programa debería funcionar igual con o sin persistencia. Así, el código es más limpio, fácil de mantener, y no tenés que andar cambiando todo si cambiás cómo se guardan los datos.

Principios SOLID

Su objetivo es mejorar la calidad del código y hacerlo más fácil de mantener y aplicar a largo plazo. Es un estándar de OO.

S - Single Responsibility Principle (Responsabilidad única): Cada clase debe hacer una sola cosa y hacerla bien.

O - Open/Closed Principle (Abierto/Cerrado): El código debe estar abierto a extensiones pero cerrado a modificaciones.

L - Liskov Substitution Principle (Sustitución de Liskov): Una subclase debe poder reemplazar a su superclase sin romper el programa. Ej: utilizar Collection en lugar de ArrayList.

I - Interface Segregation Principle (Segregación de interfaces): Es mejor tener muchas interfaces pequeñas que una grande con métodos que no siempre se usan.

D - Dependency Inversion Principle (Inversión de dependencias): El código debe depender de abstracciones, no de clases concretas.

Clase 2

ORM - Object-relational Mapping

Es una técnica que permite la integración entre una BD Relacional y un sistema OO, en vez de escribir SQL directamente.
El mapeo OO-BD relacional traduce los objetos y las relaciones de un sistema OO a un formato compatible con una BD relacional, y viceversa.

- Código más limpio y legible.
- Menos errores de SQL.
- Más rápido de escribir y mantener.

Garbage colector de doble barrido

Libera objetos de la memoria. En la 1ra pasada marca los objetos que aún están en uso, en la 2da pasada libera los objetos que no están marcados.

Mapeo de un objeto en la BD - Jerarquías de clases

Única superclase

Toda la jerarquía se guarda en una sola tabla, con una columna que indica el tipo
Pros: simple, rápida.
Contras: muchos NULL si las subclases tienen campos muy distintos.

Dejar todo

Cada subclase tiene su propia tabla, que guarda solo sus atributos.
Pros: evita NULL, respeta bien el modelo.
Contras: más compleja la consulta (hay que hacer joins con la tabla padre).

Solo clases hijas

Solo se guarda en la tabla de la subclase (la superclase no tiene tabla).
Pros: simple, una sola tabla por tipo.
Contras: duplicación si hay muchos atributos compartidos.

A veces, una sola estrategia de mapeo no alcanza para representar todo lo que la aplicación necesita. Entonces, se **combinan dos estrategias sobre el mismo modelo**. Eso puede guardar más datos, pero da más flexibilidad, y hoy en día el almacenamiento ya no es un gran problema. Se puede usar lógica de negocio o triggers para mantener la consistencia. Ejemplo:

- Usás tabla única para acceder rápido desde la app.
- Y tablas separadas por subclase para reportes más detallados o con integridad referencial más fuerte.

Hibernate

Hibernate es un framework ORM para Java. Te permite trabajar con bases de datos usando objetos Java, sin escribir SQL. Hibernate se encarga de mapear tus clases a tablas y hacer todo el trabajo de guardar, buscar, actualizar y borrar objetos en la base de datos.
Su problema es que es una herramienta, no un estándar.

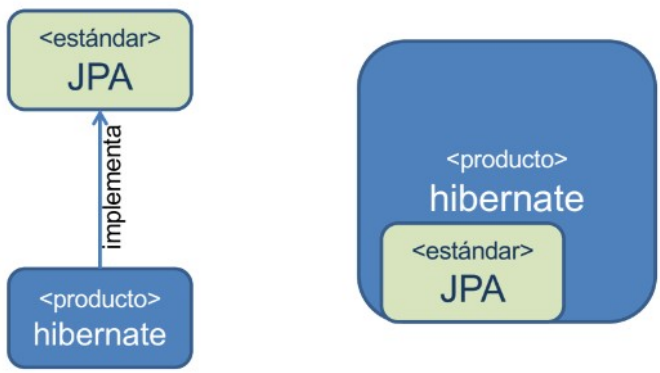
JPA (Java Persistence API)

Es un estándar de persistencia para java que permite mapear objetos Java a una BD relacional. Fue creado para simplificar el desarrollo de aplicaciones que necesitan persistir datos en una BD, proporcionando una API común para interactuar con diferentes proveedores de ORM como Hibernate, EclipseLink, etc. No es una herramienta en sí misma, sino un conjunto de reglas.

Fue diseñado para estandarizar el mapeo OO-relacional en Java, permitiendo a los desarrolladores trabajar con objetos en lugar de SQL directo, lo que facilita la portabilidad del código entre diferentes BD y proveedores ORM

Hibernate vs JPA

Hibernate es un producto y JPA es un estándar. Hibernate implementa JPA.
La consulta JPQL es siempre una consulta HQL válida. No siempre al revés.
Hibernate permite consultas más complejas porque contiene más herramientas y funciones. Es recomendable escribir las consultas en el estándar, ya que si se cambia el producto, no se cambia la consulta.



Por más que Hibernate implemente JPA, es más complejo.

XML

Es una forma de definir el mapeo entre tus clases y la base de datos usando archivos de configuración en lugar de anotaciones en el código. Se usaba mucho antes, ahora casi

todo se hace con anotaciones, pero sigue siendo válido si querés separar configuración del código.

XML vs Annotations

Anotaciones: cuando querés simplicidad y todo junto en el código. Ideal para la mayoría de los proyectos modernos.
XML: cuando querés separar configuración, cambiar mapeos sin recompilar o tener distintas versiones del mapeo (por ejemplo, en testing o migraciones).

Reglas de persistencia de POJO

Obligatorio

- Debe tener un constructor público o protegido sin argumentos

Opcional

- Debe tener un identificador único (es opcional, pero muy recomendado)
- Debe ser una clase concreta (no abstracta): Las clases abstractas no pueden ser instanciadas, y por eso no se pueden guardar directamente.
- Declarar setters y getter para los campos persistentes

Clase 3

Clase repository

Se encargan de realizar la transparencia para guardar los objetos en la BD sin tener que abrir sesiones/transacciones o configurar el acceso a la BD. Realiza todas las tareas para persistir los cambios de los elementos en la BD sin que uno sepa cómo realizan las transacciones.
Un Repository en JPA (especialmente con Spring Data JPA) es una interfaz que maneja el acceso a datos. Permite realizar el CRUD entidades sin escribir SQL.

RELACIONES en JPA/Hibernate

Cuando una entidad está relacionada con otra (por ejemplo, Banco tiene muchas Cuentas), podés controlar cómo se comporta esa relación con algunas propiedades clave, como cascade y fetch (lazy/eager).

Propiedad cascade

La propiedad cascade indica si las operaciones realizadas sobre una entidad deben propagarse a sus entidades relacionadas.
Si tenés una clase Banco que tiene una lista de Clientes. Entonces, si guardás un Banco, también se guardan automáticamente los Clientes.

Propiedad lazy

La propiedad lazy (por defecto en muchas relaciones) retrasa la carga de los datos relacionados hasta que realmente los necesitas. Solo se llaman al llamar el método get.
Para realizar esto utiliza el patrón **Proxy**: una clase “falsa” que actúa como si fuera la lista real, pero recién se conecta a la base cuando la usás.

Patrón Proxy

Es un objeto que actúa como un sustituto de otro objeto. En persistencia, los proxies se utilizan para cargar datos bajo demanda.

Cuando tienes una relación lazy, Hibernate no carga inmediatamente la entidad relacionada, sino que crea un proxy que:

- Parece ser el objeto real.
- Solo carga los datos reales de la base de datos cuando se accede por primera vez a sus propiedades (excepto el ID).

Esto optimiza el rendimiento al evitar cargar datos que podrían no necesitar.

Patrón DTO (Data Transfer Object)

Los DTO son objetos simples que transportan datos entre capas de la aplicación (por ej: entre la capa de servicio y la capa de presentación). No tiene comportamiento, solo contienen datos.

Se utilizan para reducir el número de llamadas entre capas, especialmente en aplicaciones distribuidas.

Los DTO son útiles cuando se necesita transferir solo un subconjunto de datos de un objeto o persistente, o cuando se necesita desacoplar la capa de persistencia de la capa de presentación.

Clase 4

Control de la concurrencia - Locking

Va a pasar que varios usuarios trabajen con la misma información, para eso se utilizan 2 estrategias.

Optimista

Se asume que en general no habrá demasiados problemas, y se controla usualmente con un número de versiones o timestamp.
Debe cumplir ciertas reglas:

- Cada clase debe tener exactamente un atributo versión.
- Si una clase se mapea a varias tablas debe estar en la tabla principal.

Se chequea que la versión que tiene el objeto sea la misma que la que está en la base. Cuando se intenta escribir un objeto con una versión vieja, JPA lanza una excepción indicando que los datos que se quieren escribir son inconsistentes.

Pesimista

En este escenario se evitan problemas bloqueando entidades a nivel de BD y hasta en un nivel de lectura, hasta que no se cierre una transacción no se abre otra.
Es muy raro llegar a tener que usarla porque bloquea muchos recursos.

Cuando se tienen ambientes de alta concurrencia y escalabilidad, se utiliza el control de concurrencia optimista y versionamiento.

Spring

Es un framework que facilita el desarrollo de una aplicación en JAVA otorgando múltiples funcionalidades y librerías útiles.

Clase 5

Patrón DAO (Data Access Object) [Patrón viejo]

Establece que por cada objeto de dominio persistente debe existir una clase que tenga la responsabilidad de persistir y recuperar dichas instancias. El problema con este patrón es que usualmente lleva a soluciones en las cuales el modelo es anémico (no tiene comportamiento, solo getters y setters).

Patrón Repository

Es la mejor opción para atacar la recuperación de datos (Read).

Implementa una clase que tiene todos los mecanismos de consulta de un modelo y que esa clase pueda operar con un filtro para resolver la consulta con ese criterio.

El problema con este patrón es que está pensado para un sistema que no tiene tipos, por lo que hay que trabajar mucho con la implementación de cada filtro en el repositorio. En JAVA estos filtros se traducen en métodos particulares para cada uno.

El objetivo del repository es que si se tiene que hacer una aplicación para resolver consultas, se crearía un repositorio por cada clase y que cada método del repositoriy realice las consultas necesarias a la base para devolver el resultado esperado.

Desventaja: es tedioso tener que escribir cada criterio de forma particular.
Ventaja: si se respeta el patrón, sabemos que todas las consultas se realizan en los repository y no en otra parte de la aplicación.

OQL (Object Query Language)

Es un lenguaje de consulta orientado a objetos, diseñado para consultar bases de datos que siguen el modelo de datos orientado a objetos. Es decir, se usa en sistemas donde los datos no están organizados en tablas como en SQL, sino como objetos con atributos y métodos, al estilo de la programación orientada a objetos.

- Usa una sintaxis similar a SQL, pero pensada para trabajar con objetos, jerarquías, clases, relaciones entre objetos, etc.
- Permite navegar estructuras complejas de objetos, incluyendo colecciones, listas y mapas.
- Puede invocar métodos de los objetos dentro de las consultas.

Podemos obtener todas las instancias de una clase, y a su vez las sub instancias de esa clase si está conformada por otras clases. A diferencia de SQL que solo se obtienen tablas.

HQL (Hibernate Query Language)

HQL te permite hacer consultas a nivel de objeto, no a nivel de tabla. En lugar de pensar en tablas y columnas, pensás en clases y atributos.

- Se parece mucho a SQL en la sintaxis.
- Consulta sobre entidades Java y sus relaciones, no sobre tablas y columnas.

- Soporta herencia, relaciones (uno a muchos, muchos a muchos), subconsultas, joins, etc.

Criteria (consultas dinámicas)

Es una API de Java que permite construir consultas de manera programática. Es más seguro (no hay problemas de tipeo) pero más lento.

Clase 6

Reflection

Es una API que se utiliza para examinar o modificar el comportamiento de métodos, clases e interfaces en tiempo de ejecución. Las Clases requeridas para la reflexion se proporcionan en el paquete java.lang.reflect, que es esencial para comprender la reflexion.

Reflection nos da información sobre la clase a la que pertenece un objeto y también los métodos de esa clase que se pueden ejecutar usando el objeto. A través de Reflection, podemos invocar métodos en tiempo de ejecución independientemente del especificador de acceso utilizado con ellos.

Básicamente nos sirve para no tener que hardcodear la instanciación de un objeto y la ejecución de un método, sinó que nos permite definir en tiempo de ejecución que clase se va a instanciar y que método ejecutar.

Clase 7

Bases de datos NoSQL

Están orientadas a la gestión de grandes conjuntos de datos variados y actualizados con frecuencia, a menudo en sistemas distribuidos o en nube. Evitan los esquemas rígidos asociados con las BD relacionales. Pero las arquitecturas mismas varían y están separadas en cuatro clasificaciones principales, aunque los tipos se mezclan con el tiempo.

Clave-Valor: REDIS y RIAK

No existe el concepto de tablas relacionales, son conjuntos de claves-valor. Son simples y rápidas, y se usan a menudo para aplicaciones que requieren una gran

cantidad de lectura y escritura, pero que no requieren una complejidad de consultas. Un uso común son logs de aplicaciones o para almacenar clickstream data.

Orientado a Grafos: Neo4j y GraphDB

Se utiliza para almacenar y recuperar datos con relaciones complejas y enriquecedoras, como los sistemas de recomendación, aplicaciones de IA y el análisis de redes sociales. En lugar de almacenar datos en tablas, estas representan los datos y sus relaciones mediante nodos y enlaces en un grafo.

Orientado a documentos: MongoDB

Se utilizan para almacenar y recuperar documentos estructurados, que son similares a las filas en una tabla de una BD relacional. En cada documento la estructura no es necesariamente igual ni contiene los mismos campos.

Almacena elementos de datos en estructuras similares a documentos que codifican información en formatos tipo JSON. Los usos comunes incluyen administración de contenido y monitoreo de aplicaciones web y móviles.

Familia de Columnas: Cassandra

Apache Cassandra es una BD distribuida de código abierto que se utiliza para almacenar y recuperar grandes cantidades de datos. Es altamente escalable, tolerante a fallos y se ejecuta en una arquitectura de cluster, lo que permite a los usuarios escalar horizontalmente para manejar una creciente cantidad de datos y solicitudes.

Almacena datos en tablas que pueden tener una gran cantidad de columnas. Permite a los usuarios modelar y almacenar datos de manera flexible y escalable. Cassandra es capaz de manejar grandes cantidades de datos y solicitudes de lectura y escritura simultáneamente, lo que la hace adecuada para aplicaciones que requieren un rendimiento alto y una disponibilidad confiable.

Conjetura de Brewer

También conocida como el **teorema CAP** (por *Consistency, Availability, Partition tolerance*), es un principio fundamental en el diseño de sistemas distribuidos

Eric Brewer propuso en el año 2000 que un sistema distribuido no puede garantizar simultáneamente las tres propiedades siguientes:

1. **Consistencia** (Consistency): todos los nodos del sistema ven los mismos datos al mismo tiempo. Es decir, después de una escritura exitosa, cualquier lectura posterior devolverá ese dato actualizado.
2. **Disponibilidad** (Availability): el sistema siempre responde a las solicitudes, incluso si algunas de sus partes fallan. Es decir, cada solicitud recibe una respuesta, sin garantía de que sea la más actual.
3. **Tolerancia a particiones** (Partition tolerance): el sistema sigue funcionando a pesar de fallas o pérdida de comunicación entre partes del sistema (particiones de red).

En presencia de una partición en la red, un sistema distribuido debe elegir entre ser consistente o estar disponible. Esto no significa que no puedas tener un poco de los tres, pero no podés garantizar completamente los tres al mismo tiempo.

ACID

Este modelo garantiza que las transacciones se manejen de forma segura y confiable. tradicional, usado en bases de datos relacionales
A – Atomicidad: la transacción se ejecuta toda o nada. Si algo falla, se revierte todo.
C – Consistencia: la base de datos pasa de un estado válido a otro. No queda nunca en un estado “raro”.
I – Aislamiento: las transacciones concurrentes no se interfieren entre sí. Se comportan como si se ejecutarán de a una.
D – Durabilidad: una vez que la transacción se completa, los cambios quedan guardados, incluso si hay un corte de luz.

BASE

Este modelo aparece con las bases de datos distribuidas y NoSQL, donde se prioriza la escalabilidad y disponibilidad, sacrificando un poco de consistencia.
BA – Basically Available: el sistema siempre responde, aunque a veces con datos viejos o incompletos.
S – Soft state: el estado del sistema puede cambiar con el tiempo, incluso sin nuevas escrituras (por replicación, por ejemplo). Es decir que los datos pueden estar en constante cambio.
E – Eventually consistent: eventualmente, todos los nodos tendrán los mismos datos, pero no de inmediato.

BSON

Es un formato de codificación para documentos tipo JSON. Agarra el JSON que se envía, hace una validación de que sea un JSON válido y lo traduce a JSON Binario, llamado BSON. Tienen las siguientes características:

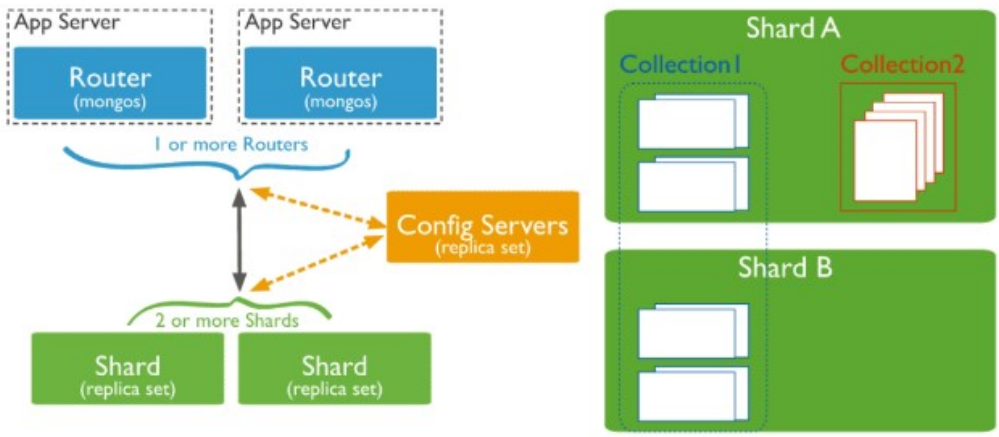
- Liviano: mantiene el mínimo espacio.
- Recorrible: debe ser fácilmente recorrible.
- Eficiente: la codificación y decodificación puede ser realizada muy rápidamente ya que se basa en tipos de datos de C.

Sharding

Es una técnica de partición de BD que permite dividirla en múltiples partes, llamadas shards, para mejorar el rendimiento y la escalabilidad de un sistema. Cada Shard es una BD independiente y autónoma, que puede ser almacenada en un servidor diferente y gestionada de manera separada.

El objetivo principal es distribuir la carga de trabajo, y los datos de manera equilibrada entre varios servidores, lo que permite que el sistema sea más eficiente y escalable. Con el tiempo, el crecimiento del volumen de datos y la demanda de rendimiento, los shards pueden ser agregados o eliminados para mantener un equilibrio adecuado.

Crecimiento vertical: implica aumentar las capacidades de un solo servidor.
Crecimiento horizontal: implica dividir la información en porciones más pequeñas y distribuir la carga en múltiples equipos.



Shard Key

La shard key es una clave o campo específico en una colección que se utiliza para determinar a qué shard pertenece cada documento en la colección. La shard key es esencial en MongoDB ya que permite la distribución de los datos de manera equilibrada entre los shards, lo que mejora el rendimiento y la escalabilidad del sistema de base de datos. Es importante elegir una shard key que tenga una distribución uniforme de los

valores para garantizar una distribución equilibrada de los datos entre los shards y evitar una sobrecarga en un solo shard.

Clase 10

ElasticSearch

Es un motor de búsqueda y análisis de datos en tiempo real basado en la tecnología de Apache Lucene. Provee interfaz RESTful. Se usa para indexar, buscar y analizar grandes cantidades de datos en tiempo real. No está pensado para ser una BD.

Utilizado una arquitectura de microservicios, si por ejemplo tengo un Oracle con mucha información, y se hace difícil buscar un string utilizando selects entre todas las tablas, entonces lo que se hace es tener un worker que replique todo el tiempo la información del Oracle en una estructura como ElasticSearch. Permite realizar búsquedas de ese string de una forma más eficiente y performante entre las tablas que habían en Oracle.

Docker

Es una tecnología que permite crear, ejecutar y gestionar contenedores, que son entornos aislados y livianos que comparten el kernel del sistema operativo host. Cada contenedor corre una imagen, que es una plantilla inmutable construida a partir de un archivo llamado Dockerfile, donde se especifica qué software, configuraciones y comandos debe tener ese contenedor. Aunque los contenedores son volátiles (si se eliminan, se pierde su estado interno), se pueden usar volúmenes para guardar datos de manera persistente en el sistema host. Esto es clave cuando se trabaja con bases de datos, ya que permite que los datos sobrevivan aunque el contenedor se reinicie o destruya. Además, Docker permite tener múltiples contenedores creados a partir de una misma imagen, lo que facilita la escalabilidad horizontal y la separación de ambientes.

HaProxy

Es un software que funciona como balanceador de carga y proxy inverso, recibiendo las solicitudes de los clientes y distribuyéndolas entre varios servidores backend, como si fuera un intermediario inteligente. Esto permite mejorar la disponibilidad y escalabilidad de una aplicación, ya que evita que todo el tráfico llegue a un único servidor.

Clase 11

JDO (Java Data Objects)

Es una especificación basada en interfaces que permite la persistencia en objetos Java, funcionando de forma transparente respecto al tipo de almacenamiento subyacente (puede ser archivos, XML, BD relacionales o BD OO). Su objetivo es ofrecer una forma estándar de almacenar y recuperar objetos sin imponer restricciones significativas sobre cómo se escriben las clases Jaca ni requerir un lenguaje de consulta nuevo. A diferencia de otros enfoques, no reemplaza JDBC, sino que ofrece una abstracción de más alto nivel, OO, sobre el almacenamiento persistente.

JDO permite persistir objetos Java sin tener que convertirlos manualmente a un modelo relacional. Soporta operaciones CRUD de manera automática y transparente, y permite consultas usando mecanismo basado en Java. Su modelo de datos es completamente OO, soporta transacciones, y está diseñado para ser portable y compatible entre distintas implementaciones. Además, está basado en metadatos, lo que facilita el mapeo y manejo automático de las entidades.

Serialización

Se refiere al proceso de convertir un objeto en secuencia de bytes o texto que puede ser almacenada o transmitida. Es una funcionalidad estándar de Java, fácil de usar, y permite enviar objetos por red o guardarlos en disco. Sin embargo, no está pensada como mecanismo de persistencia real, ya que no es escalable, no soporta consultas complejas ni operaciones eficientes como las BD. Además, serializar solo convierte los datos, no los estructura ni permite acceso parcial o por criterios como lo haría una BD.

JDBC (Java Database Connectivity)

Es una API que permite conectar Java con BD relacionales mediante SQL. Usa un modelo relacional clásico, como tablas, filas y columnas, y se basa en operaciones SQL escritas por el programador. JDBC no trabaja directamente con objetos Java como tal, por lo que se requiere transformar manualmente los resultados SQL a objetos y viceversa. Tiende más al paradigma procedural y las implementaciones pueden no ser portables entre distintos motores de BD.

JDO vs JDBC

La principal diferencia es el paradigma: JDBC está enfocado en el modelo relacional y en el uso directo de SQL, mientras que JDO trabaja desde el paradigma OO, permitiendo persistir y consultar objetos de forma más natural desde Java.

JDO vs Hibernate

Aunque ambos permiten la persistencia de objetos de Java, su funcionamiento interno es distinto. Hibernate analiza todo código, mira los mapeos y realiza la generación de clases dinámicamente en tiempo de ejecución, lo que hace que al levantar un servidor tarde más, ya que debe interpretar todos los mapeos en ese momento. JDO, en cambio, realiza este análisis en tiempo de compilación, por lo que la carga del servidor es más rápida. Sin embargo, una desventaja de JDO es que , al compilar clases con mucho procesamiento automático, los errores que surgen en tiempo de ejecución pueden ser difíciles de rastrear, ya que el código generado tiene muchas más líneas que el código fuente original.

Archivos .class aumentados

En JDO, los archivos .class aumentados son versiones especiales de las clases Java que ya vienen preparadas para poder guardar objetos en una base de datos. A diferencia del .class normal (que solo tiene el código compilado original), estos archivos aumentados incluyen código extra que JDO necesita para manejar la persistencia de los objetos.

Sirven para que las clases puedan ser guardadas, modificadas o recuperadas automáticamente sin que el programador tenga que escribir todo ese código manualmente. Gracias a esto, JDO puede saber cuándo un objeto cambió, controlar transacciones, y trabajar con la BD sin complicaciones.

Una ventaja importante es que todo esto se hace antes de ejecutar la aplicación, por lo que el sistema arranca más rápido en comparación con otros enfoques que hacen esta transformación en tiempo de ejecución, como Hibernate.

Preguntas de final

Estado de JDO (ej: transient, persistent, detached)
Transient: el objeto no está asociado con una transacción y no es persistente.
Persistent: el objeto está asociado con una transacción y es persistente.
Detached: el objeto fue persistente pero ya no está asociado con una transacción.

Ejemplos de inserción, búsqueda y eliminación en mongoDB

Insertar varios documentos:

```
db.usuarios.insertMany([ { nombre: "Juan", edad: 25, email: "juan@mail.com" },]);
```

Búsqueda

```
db.usuarios.findOne({ nombre: "Ana" });
```

Eliminar

```
db.usuarios.deleteOne({ nombre: "Ana" });
```

Reemplaza

```
db.usuarios.replaceOne(
  { nombre: "Ana" },
  { nombre: "Ana", edad: 32, email: "ana_nueva@mail.com" }
);
```

Para que se usan los detached objects?

Un objeto que fue recuperado en la BD en una sesión anterior, pero que ya no está asociado a una sesión activa. Es decir, está 'desconectado' del contexto de persistencia. Se usan para:

- evitar consultas innecesarias: si ya consultaste un objeto y solo querés trabajar con sus datos sin volver a la base, podés usarlo como detached.
- Escenarios distribuidos o en varias capas: en aplicaciones con arquitectura por capas (back, front, ...), muchas veces se pasa un objeto ya cargado (detached) del back al front, se modifica, y luego se vuelve a adjuntar para guardar los cambios.
- Persistir cambios luego: Podés modificar un detached object y más adelante relanzarlo a una sesión para sincronizar los cambios con la base.

Cual es la finalidad de "discriminator-value" en el mapeo de un modelo?

Se usa para distinguir entre subclases en una jerarquía de herencia mapeada a una sola tabla en la BD. Es decir que cuando se usa herencia en un modelo, se puede mapear todas las clases a una sola tabla (estrategia llamada Single Table Inheritance). Ej: una clase Empleado con subclases Gerente, Técnico, etc. Acá entra el discrimination-value, sirve para identificar a qué subclase pertenece cada fila de la tabla.

¿Cuál es el final del patrón Root Object? Y qué relación tiene con discrimination-value en mapeo.

La finalidad es modelar el sistema como un objeto en sí mismo, lo que facilita la gestión de la persistencia y las relaciones entre objetos.

La relación con discrimination-value está en el mapeo de jerarquías, ya que d-v se usa para distinguir entre diferentes tipos de objetos en una tabla única. El Root object puede ser el punto de entrada para acceder a estos objetos.

Describir un breve modelo e indicar cómo se realiza una consulta en JDOQL, en forma de string y declarativa.

Modelo:

```
@PersistenceCapable
public class Persona {
   String nombre;
   int edad;
}
```

Forma string (dinámica):

```
String queryStr = "SELECT FROM Persona WHERE edad > 30";
Query query = pm.newQuery(queryStr);
query.execute();
```

Explicar Union-Subclass.

Es una estrategia de mapeo de herencia usada en ORM donde cada subclase se mapea su propia tabla y la superclase a otra tabla separada. Cuando consultás unas subclases, se hace un JOIN entre la tabla de la subclase y la tabla de la superclase.

¿Qué atributo "fuerza" la desvinculación de objetos persistentes en una base de datos?

Cuando se trabaja con objetos persistentes, estos se mantienen conectados al contexto de persistencia. Pero si se activa detachAllOnCommit todos los objetos persistentes se desconectan automáticamente del contexto.

Spring Data, que es?

¿Por defecto, qué métodos ofrece?

¿Qué formas hay de declarar una query a la base utilizando Spring Data? Explicar cada una de ellas.

Es un módulo del framework Spring que tiene como objetivo simplificar el acceso a BD y reducir la cantidad de código necesario para implementar la capa de persistencia. Permite trabajar de forma muy sencilla con distintas tecnologías de almacenamiento de datos, mediante la definición de interfaces que Spring implementa automáticamente.

Por defecto, Spring Data ofrece una serie de métodos básicos que permiten realizar operaciones comunes sin necesidad de escribir consultas explícitas. Entre estos métodos se encuentran los necesarios para guardar o actualizar una entidad, buscar por ID, listar todos los registros, eliminar por ID o por instancia, contar cuantos registros existen o verificar si existe una entidad con cierto ID. Todos estos métodos se encuentran disponibles al extender interfaces como CrudRepository o JpaRepository.

Para declarar una consulta se hace mediante el nombre del método. Spring Data interpreta el nombre de método y genera automáticamente la consulta correspondiente.
También se puede utilizar la anotación @Query. Esta permite escribir directamente la consulta usando JPQL.

Al cambiar el mecanismo de persistencia de un relacional a un nosql, ¿Qué capas se deberían modificar? Explicar.

Capas que no deberían cambiar:

- lógica de negocio
- interfaz de usuario

Capas a modificar

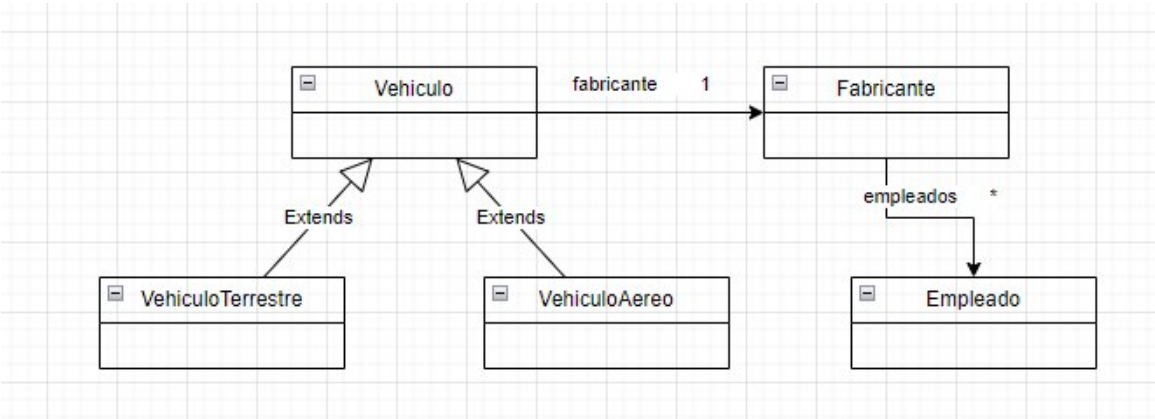
- capa de persistencia (DAO)
    - cambia la forma de acceder a los datos
    - se reemplazan consultas SQL por la lógica de NoSQL
- configuración de la conexión
    - cambiar driver, url, autenticación y proveedores de persistencia
- modelo de datos (opcional)

Características de OQL y diferencia con SQL. De un ejemplo que muestra la potencia de OQL

Características OQL

- consulta objetos en lugar de registros/tablas
- permite navegar por atributos y referencias entre objetos (como listas dentro de la clase)
- se integra a lenguajes de programación
- soporta jerarquías de herencia, polimorfismo y relaciones complejas
- no necesita tener solo datos primitivos, tiene atributos complejos como listas y maps

Listar todos los VehiculosAeres cuyo fabricante tenga al menos 100 empleados



SQL	OQL (Object Query Language)

SELECT va.*
FROM vehiculo_aereo va
JOIN vehiculo v ON va.id = v.id
JOIN fabricante f ON v.fabricante_id = f.id
WHERE (
  SELECT COUNT(*)
  FROM empleado e
  WHERE e.fabricante_id = f.id
) >= 100;
	SELECT va
	FROM VehiculoAereo va
	WHERE va.fabricante.empleados.size >= 100

HQL (Hibernate Query Language)	**JDOQL (Java Data Objects Query Language)**
FROM VehiculoAereo va WHERE size(va.fabricante.empleados) >= 100	SELECT FROM VehiculoAereo WHERE this.fabricante.empleados.size() >= 100